Functions

Functions are reusable blocks of code that perform a specific task. They help in organizing code, improving readability, and reducing redundancy. Functions help break down complex programs into smaller, more manageable pieces, making code easier to read, write, and maintain. Python treats functions as first-class objects, meaning they can be assigned to variables, passed to other functions and returned as values.

There are three types of functions in Python:

- Built-in functions, such as `help()` to ask for help, `min()` to get the minimum value, `print()` to print an object to the terminal.
- User-Defined Functions (UDFs), which are functions that users create to help them out; And
- Anonymous functions, which are also called lambda functions because they are not declared with the standard `def` keyword.

Defining a Function

Defining a function involves creating a reusable block of code that performs a specific task. Functions are defined using the `def` keyword, followed by the function name, a pair of parentheses `()`, and a colon:. The block of code inside the function is indented and executed when the function is called.

Syntax

```
def function_name(parameters):
    # Function body
    # Code to execute
```

- **def**: Keyword used to define a function.
- **function_name**: Name of the function (follows the same rules as variable names).
- **Parameters**: Optional inputs to the function (enclosed in parentheses).
- **Function body**: The indented block of code that defines what the function does.
- **Return Value**: Functions can return a value using the `return` statement.

If no `return` statement is provided, the function returns **None**.

Calling a Function

Calling a function means executing the code defined within the function. Defining a function only gives it a name, specifies the parameters that are to be included in the function and structures the blocks of code. Once the basic structure of a function is finalized, you can call it by using the function name itself. If the function requires any parameters, they should be passed within parentheses. If the function doesn't require any parameters, the parentheses should be left empty.

Syntax

```
function_name(arguments)
```

- **function_name**: Name of the function to call.
- **arguments**: Values passed to the function (if it accepts parameters).
- If the function has parameters, provide the required arguments in the correct order.
- Arguments can be positional, keyword, or default.
- **return value**: If the function returns a value, you can store it in a variable or use it directly.

Pass by Reference vs Value

call by value − When a variable is passed to a function while calling, the value of actual arguments is copied to the variables representing the formal arguments. Thus, any changes in formal arguments does not get reflected in the actual argument. This way of passing variable is known as call by value.

call by reference − In this way of passing variable, a reference to the object in memory is passed. Both the formal arguments and the actual arguments (variables in the calling code) refer to the same object. Hence, any changes in formal arguments does get reflected in the actual argument.

Python uses pass by reference mechanism. As variable in Python is a label or reference to the object in the memory, both the variables used as actual argument as well as formal arguments really refer to the same object in the memory. We can verify this fact by checking the id() of the passed variable before and after passing.

```
def testfunction(arg):
        print ("ID inside the function:", id(arg))
var = "Hello"
print ("ID before passing:", id(var))
testfunction(var)
```

Function Parameters

A function with parameters is defined to accept one or more inputs, called parameters, which act as placeholders for the data passed to the function when it is called. These parameters allow the function to perform operations based on the provided inputs.

Parameters are variables that are defined in the function definition. They are assigned the values which were passed as arguments when the function was called, elsewhere in the code.

Syntax:

```
def function_name(parameter1, parameter2, ...):
    # Function body (code block)
    # Use parameters in operations
    return result  # Optional
```

Example:

```
def greet(name):
    message = f"Hello, {name}!"
    return message
# Calling the function
print(greet("Alice"))  # Output: Hello, Alice!
print(greet("Bob"))    # Output: Hello, Bob!
```

Python Function Arguments

Function arguments are the values or variables passed into a function when it is called. The behavior of a function often depends on the arguments passed to it.

While defining a function, you specify a list of variables (known as formal parameters) within the parentheses. These parameters act as placeholders for the data that will be passed to the function when it is called. When the function is called, value to each of the formal arguments must be provided. Those are called actual arguments.

Formal and Actual Arguments

Formal Arguments (Parameters)

Formal arguments are the variables listed in a function's definition. They act as placeholders that receive values when the function is called. These are also called **parameters**.

They define the input structure of the function and are used within the function to perform computations or operations.

Characteristics:

- Formal arguments are local to the function's scope, meaning they cannot be accessed outside the function.
- They are defined in the function signature, e.g., def function_name(param1, param2):.
- The number and type of formal arguments determine how the function must be called.

Example:

```
def multiply(x, y):  # 'x' and 'y' are formal arguments
    return x * y
```

Here, x and y are formal arguments that expect two values when the function is called.

Edge Case:

If a formal argument is defined but not provided during the function call (and has no default value), Python raises a TypeError.

```
multiply(5)  # TypeError: multiply() missing 1 required positional argument:
'y'
```

Actual Arguments

Actual arguments are the real values or expressions passed to a function during its invocation. They are mapped to the formal arguments.

They provide the data that the function will process.

- **Characteristics**:
  - Actual arguments can be literals (e.g., 5, "Hello"), variables, or expressions.
  - They must match the number and type expected by the formal arguments unless defaults or variable-length arguments are used.

Example:

```
result = multiply(4, 5)  # '4' and '5' are actual arguments
print(result)  # Output: 20
```

Here, 4 is assigned to x, and 5 is assigned to y.

Edge Case:

- Passing too many actual arguments causes a TypeError unless the function supports variable-length arguments (*args).

```
multiply(4, 5, 6)  # TypeError: multiply() takes 2 positional arguments but 3 were given
```

Positional Arguments

Positional arguments are arguments passed to a function in the same order as the formal parameters are defined in the function signature.

They are the simplest way to pass arguments, relying on the order of parameters to map values correctly.

- **Characteristics**:
  - The position of each actual argument corresponds to the position of the formal parameter.
  - They are mandatory unless the parameter has a default value.

Example:

```
def student_info(name, age, grade):
    print(f"Name: {name}, Age: {age}, Grade: {grade}")
student_info("Alice", 15, "10th")  # Output: Name: Alice, Age: 15, Grade: 10th
```

Here, "Alice" maps to name, 15 to age, and "10th" to grade based on position.

- **Edge Case**:
  - If the number of positional arguments doesn't match the number of formal parameters, Python raises a TypeError.

```
student_info("Alice", 15)  # TypeError: student_info() missing 1 required positional argument: 'grade'
```

Keyword Arguments

Keyword arguments are passed to a function by explicitly specifying the parameter name and its value (e.g., param=value) during the function call.

They improve code readability and allow arguments to be passed in any order, avoiding errors due to position mismatches.

- **Characteristics**:
  - The parameter name is used to map the value, making the order irrelevant.

    - They are often used when a function has many parameters or when some arguments are optional.
- **Example**:

```
def book_details(title, author, year):
    print(f"Book: {title}, Author: {author}, Year: {year}")

book_details(author="J.K. Rowling", title="Harry Potter", year=1997)
# Output: Book: Harry Potter, Author: J.K. Rowling, Year: 1997
```

- **Edge Case**:
    - If a keyword argument is passed for a non-existent parameter, Python raises a TypeError.

```
book_details(title="Harry Potter", writer="J.K. Rowling")  # TypeError: book_details() got an unexpected keyword argument 'writer'
```

- **Advantages**:
    - Enhances readability, especially for functions with many parameters.
    - Reduces errors caused by incorrect argument order.
    - Useful in libraries or APIs where function signatures are complex.

Default Arguments in Python

- Default arguments are values that are provided while defining functions.
- The assignment operator = is used to assign a default value to the argument.
- Default arguments become optional during the function calls.
- If we provide a value to the default arguments during function calls, it overrides the default value.
- The function can have any number of default arguments.
- Default arguments should follow non-default arguments.

In the below example, the default value is given to argument b and c.

```
def add(a,b=5,c=10):

    return (a+b+c)
```

This function can be called in one of three ways:

Giving Only the Mandatory Argument

```
print(add(3))

#Output:18
```

Giving One of the Optional Arguments

3 is assigned to a, 4 is assigned to b.

```
print(add(3,4))

#Output:17
```

Giving All the Arguments

```
print(add(2,3,4))

#Output:9
```

Default values are evaluated only once at the point of the function definition in the defining scope. So, it makes a difference when we pass mutable objects like a list or dictionary as default values.

Variable Length Arguments

Variable-length arguments allow a function to accept an arbitrary number of arguments, either as positional arguments (*args) or keyword arguments (**kwargs).

They provide flexibility when the number of arguments is unknown or varies.

- **Types**:
  - ***args**: Collects extra positional arguments into a tuple.
  - ****kwargs**: Collects extra keyword arguments into a dictionary.

*Non-Keyword Variable-Length Arguments (*args)*

The *args syntax allows a function to accept any number of positional arguments, which are stored in a tuple named args (or any name preceded by *).

Characteristics:

- The * unpacks the arguments into a tuple.
- Useful for functions that need to handle a variable number of inputs, like summing numbers or concatenating strings.

Example:

```
def average(*args):
    if not args:  # Check if args is empty
        return 0
    return sum(args) / len(args)

print(average(10, 20, 30))       # Output: 20.0
print(average(1, 2, 3, 4, 5))   # Output: 3.0
print(average())                 # Output: 0
```

- **Edge Case**:
  - If *args is used, it must appear **after regular and default parameters** in the function definition.

```
def func(a, *args, b):  # SyntaxError: *args must come after all positional
parameters
    pass
```

*Keyword Variable-Length Arguments (**kwargs)*

The **kwargs syntax allows a function to accept any number of keyword arguments, which are stored in a dictionary named kwargs (or any name preceded by **).

Characteristics:

- The ** unpacks the keyword arguments into a dictionary where keys are parameter names and values are the arguments.
- Useful for passing configuration options or metadata.

Example:

```
def user_profile(**kwargs):
    for key, value in kwargs.items():
        print(f"{key}: {value}")

user_profile(name="Alice", age=25, city="London", job="Engineer")
```

- **Combining All Argument Types**:
  - You can combine regular, default, *args, and **kwargs in a function, but they must follow this order:
    1. Regular positional parameters
    2. Default parameters
    3. *args
    4. **kwargs

    Example:

    ```
    def complex_function(a, b=10, *args, **kwargs):
        print(f"Regular: {a}")
        print(f"Default: {b}")
        print(f"Positional extras: {args}")
        print(f"Keyword extras: {kwargs}")

    complex_function(1, 2, 3, 4, x="extra", y="data")
    ```

Return Values and `return` Statements in Python

In Python, **return values** and the **`return` statement** are fundamental parts of defining and using functions. They allow a function to send data back to the part of the program where it was called. This is useful when a function performs a computation or task and needs to return a result.

What is a Return Value?

A **return value** is the output that a function sends back to the caller after execution. This output can be:

- A single value like a number or string
- Complex types such as lists, dictionaries, or even other functions
- A group of values returned together as a tuple

For example:

```
def add(a, b):
    return a + b

result = add(3, 4)  # result now holds the value 7
```

In this example, `add()` returns the sum of `a` and `b`, which is then stored in the variable `result`.

What is a `return` Statement?

The **`return` statement** is used inside a function to:

- Exit the function immediately
- Pass the specified value back to the caller

Once a `return` statement is executed, no further code in the function is executed. This makes `return` useful for exiting a function early, especially based on a condition.

Syntax of `return` Statement

```
def function_name(parameters):
    # Function body
    return value
```

You can store the returned result like this:

```
variable = function_name(arguments)
```

Key Concepts to Remember

- **Single Return Value**:
  A function can return a single result (like `return x + y`).
- **Multiple Return Values**:
  A function can return multiple values by separating them with commas. Python automatically packs these into a tuple.
  Example:

  ```
  return x, y  # returns a tuple (x, y)
  ```

- **No Return Statement**:
  If a function doesn't have a `return` statement, Python automatically returns `None`.
  Example:

  ```
  def greet():
      print("Hello!")  # No return value
  ```

- **Early Exit**:
  The `return` statement can be used to stop function execution early.
  Example:

  ```
  def check_even(n):
      if n % 2 == 0:
          return True
      return False
  ```

Why `return` Is Important

The use of `return` makes functions more useful and modular. By returning values, functions can pass results around in a program, making code easier to manage, reuse, and test.

The None Value in Python

`None` is a **special constant** in Python that represents the **absence of a value** or a **null value**. It is commonly used when a variable or function does not hold any meaningful value or result. `None` is an object of the **`NoneType`** class and is the **only instance** of that class in Python.

Characteristics of None

1. Singleton Object

- Python has only **one `None` object**, and it is **immutable** (cannot be changed).
- Because there is only one `None`, it is safe to compare with `is` (identity check) instead of `==`.

```
x = None
if x is None:
    print("x has no value")
```

2. Default Return Value

- If a function does **not explicitly return** a value using a `return` statement, Python **automatically returns `None`.**

```
def greet():
    print("Hello")

result = greet()      # prints "Hello"
print(result)         # prints None
```

3. Placeholder Use

- `None` is often used as a **placeholder** to indicate that a variable exists but has not been assigned a value yet.
- This is useful in function defaults and when building data structures gradually.

```
def show_message(msg=None):
    if msg is None:
        print("No message provided.")
    else:
        print("Message:", msg)
```

4. Boolean Context

- `None` is considered **falsy** in Python. This means it behaves like `False` in conditional checks.

```
if not None:
    print("None is treated as False")
```

When to Use None

- As a **default return value** when there's nothing to return from a function.
- As a **placeholder** for optional parameters or uninitialized variables.
- In **comparisons** to detect the absence of a value (always use `is` instead of `==`).
- To signal special cases like errors, missing data, or "do nothing" behavior.

Remember

- `None` is **not the same** as `0`, `False`, an empty string `''`, or an empty list `[]`.
  All of those represent *something*, whereas `None` represents *nothing*.
- `None` is **not printed** unless you explicitly print it or store it in a variable.

Understanding Local and Global Scope in Python

In programming, **scope** refers to the context in which a variable or name is defined and accessed. It tells you **where a variable can be used** and **how long it exists** in memory. Python has well-defined rules for variable scope to help you manage data safely and predictably.

There are two main types of scope in Python:

🔼 1. Global Scope

A **global variable** is a variable that is defined **outside of all functions** and is accessible throughout the file or module. These variables can be used **inside functions** (read-only by default) and **outside functions** (read and write).

Characteristics:

- Created outside of all functions.
- Accessible from **any part of the code**, including inside functions.
- If you want to **modify** it inside a function, you must declare it with the `global` keyword.

Example: Accessing a Global Variable

```
x = 10  # global variable

def show():
    print("Inside function:", x)

show()
print("Outside function:", x)
```

Output:

```
Inside function: 10
Outside function: 10
```

Here, `x` is accessible both inside and outside the function because it is in the global scope.

Example: Modifying a Global Variable Inside a Function

```
x = 5

def update():
    global x  # tells Python to use the global variable x
    x = 20

update()
print(x)  # Output: 20
```

If you don't use `global`, Python will treat `x` inside the function as a **new local variable**, leaving the global `x` unchanged.

🔼 2. Local Scope

A **local variable** is one that is **defined inside a function** and only exists **within that function**. It is created when the function starts and destroyed when the function ends.

Characteristics:

- Created and used **only inside** the function where it's defined.
- **Cannot** be accessed outside the function.
- Takes priority over any global variable with the same name (local shadowing).

Example: Using a Local Variable

```
def greet():
    name = "Aadil"  # local variable
    print("Hello", name)

greet()
print(name)  # ✗ Error: name is not defined
```

Here, `name` only exists **inside** the function. Once `greet()` is done, the variable is **destroyed**.

Example: Local Variable Shadows Global Variable

```
x = "Global"

def test():
    x = "Local"
    print("Inside function:", x)

test()
print("Outside function:", x)
```

Output:

```
Inside function: Local
Outside function: Global
```

The local variable `x` **overrides** the global `x` **within the function**. Outside the function, the global variable is still used.

The global Statement in Python

In Python, the global statement is used **inside a function** to **refer to a variable that is defined in the global scope**. Without this keyword, any assignment to a variable inside a function will by default create a **new local variable**, even if a variable with the same name exists globally.

The global keyword tells Python not to create a new local variable, but instead to use and modify the globally defined variable.

Why Do We Need global?

By default, variables defined inside a function are **local to that function**. This means you can read global variables inside functions, but if you try to **modify** them, Python will treat them as **local unless you use** global.

Example Without global (local variable created):

```
x = 10

def change():
    x = 20   # creates a new local variable, doesn't
affect global x
    print("Inside function:", x)

change()
print("Outside function:", x)
```

Output:

```
Inside function: 20
Outside function: 10
```

Here, `x = 20` inside the function creates a **new local variable** named `x`. The global `x` remains unchanged.

Example With global (global variable modified):

```
x = 10

def change():
    global x
    x = 20   # modifies the global variable
    print("Inside function:", x)

change()
print("Outside function:", x)
```

Output:

```
Inside function: 20
Outside function: 20
```

Now, `global x` tells Python to use the `x` defined outside the function, so the change affects both inside and outside values.

📌 Syntax of `global`

```
global variable_name
```

You can even use it for **multiple variables**:

```
global a, b, c
```

Important Points

- The `global` statement must appear **before the variable is used or assigned** in the function.
- You **cannot use `global` to declare new variables only inside a function**—it's meant to **refer to variables that exist in the global scope**.
- If the global variable doesn't already exist, using `global` will **create it in the global namespace**.

❌ When You Should Avoid global

While `global` can be helpful, it's often **discouraged in large programs** because:

- It makes debugging harder (global state can be changed unexpectedly).
- It reduces the **modularity** of functions.

- It can lead to **side effects** and unpredictable behavior.

Instead, consider using **function parameters**, **return values**, or **classes/objects** to manage state.